Spanner: Paper Review
Rohan Gore - rmg9725

Collaborators: ChatGPT 5

References

1. "Spanner: Google's Globally-Distributed Database" (original paper)

1 Problem Statements Tackled

1. Google required a globally distributed database that could store data across multiple continents while still providing strong, externally consistent transactions.
2. Existing systems either offered high availability with weak consistency (e.g., Dynamo, Cassandra) or strong consistency but only within a single datacenter (e.g., traditional RDBMS systems).
3. Synchronous replication across continents traditionally leads to high latency, making it difficult to offer both low-latency access and strict correctness guarantees.
4. Google needed a system where commit order reflects real-world time order, something impossible with traditional distributed clocks due to clock drift and skew.
5. A solution was required that enabled SQL semantics, automatic sharding, synchronous replication, and the ability to serve both OLTP and analytical queries without sacrificing correctness.

2 Key Design Principles

1. **External Consistency via TrueTime:** Spanner enforces external consistency (strict serializability) by assigning commit timestamps that reflect real-time order. This is made possible by exposing bounded time uncertainty through the `TrueTime` API and enforcing a "commit-wait" period before making writes visible.

2. **Structured Data + Automatic Sharding** Spanner supports a semi-relational schema, including tables and secondary indexes. Data is automatically partitioned into *directories*, which can be independently moved or resharded for load balancing.

3. **Lock-Based Read-Write Transactions + MVCC Reads** Write transactions use two-phase locking and two-phase commit across Paxos groups. Read-only transactions use multi-version timestamps, requiring no locks, enabling high-performance snapshot reads from followers.

4. **Decoupling of Time, Concurrency, and Replication** By separating timestamp selection from replica agreement, Spanner avoids the need for total-order broadcast and instead uses physical time ordering enforced through TrueTime.

3 Architecture and Components

3.1 TrueTime API, Commit Protocol, and Spanserver Stack

TrueTime returns an interval ([earliest, latest]) that reflects bounded clock uncertainty. A transaction commits at timestamp $s \geq TT.now().latest$ and is not externally visible until `TT.after(s)` holds, enforcing real-time ordering. At the implementation layer, each spanserver runs a stack consisting of: (i) Colossus files for tablet state and write-ahead logging, (ii) a pipelined Paxos state machine per tablet with 10-second leader leases, (iii) a lock table for 2PL at leaders, and (iv) a transaction manager that coordinates 2PC when multiple Paxos groups are involved. Writes enter Paxos at the leader and are applied in log order; reads may go to any replica whose safe time permits.

3.2 Paxos-Based Replication and Leader Leases

Leaders obtain timed lease votes (default 10s). Successful writes implicitly extend votes. Define a leader's lease interval from when it gains a quorum of lease votes until quorum expires. Spanner enforces the disjointness invariant: within a Paxos group, leader lease intervals never overlap. Leaders may abdicate by releasing votes, but must first wait until TT.after(s_{max}), where s_{max} is the maximum timestamp they have assigned, to preserve cross-leader timestamp monotonicity. Writes are ordered and applied by Paxos; reads may be served by any replica that is sufficiently up-to-date.

3.3 Directories and Dynamic Resharding

A directory is a contiguous key-prefix bucket and the unit of placement and replication policy. A Paxos group may contain multiple directories; thus a Spanner tablet can encapsulate disjoint key intervals, enabling co-location of frequently co-accessed directories. Live migration uses extttmovedir: bulk-copy in the background, then an atomic transactional flip of the final delta and metadata update. Large directories are transparently sharded into fragments; extttmovedir moves fragments, enabling fine-grained rebalancing and geo-placement.

3.4 Read-Only and Snapshot Transactions

extbfOperation types. RW transactions use pessimistic locking at leaders; RO transactions are predeclared, lock-free snapshot transactions; snapshot reads are single-statement reads at a past timestamp (client-specified timestamp or staleness bound). extbfTimestamp assignment. Avoid false blocking by tracking fine-grained safe times and commit times per key-range in the lock table.

3.5 Schema, Interleaving, and Index Management

Spanner supports DDL and imposes hierarchical locality via `INTERLEAVE IN PARENT`. This binds child table rows into the same keyspace as their parent rows, improving co-location and reducing cross-shard joins. Secondary indexes are fully transactional and replicated.

4 Related Work

1. **Megastore**: Introduced synchronous replication and semi-relational schemas, but suffered from high latency due to per-write Paxos and lack of stable leaders.
2. **DynamoDB**: Key–value store focused on availability and eventual consistency within a single region, unlike Spanner's global external consistency.
3. **Scatter & Gray–Lamport**: Earlier work on layering transactions over replicated stores, but with higher commit overhead than Spanner's TrueTime-based 2PC over Paxos.
4. **Calvin / H-Store / Granola**: Reduced locking through deterministic or partitioned execution, but did not provide external consistency across datacenters.
5. **VoltDB & NewSQL systems**: Scaled transactional SQL via sharding, but relied on master–slave replication and lacked the flexible geo-replication and timestamp guarantees enabled by TrueTime.

5 Conclusion

Spanner is the first production system to provide globally distributed, synchronously replicated, serializable transactions with real-time correctness guarantees. Its core contribution, the TrueTime API, enables the system to safely reason about time in a distributed setting, making external consistency practical at planetary scale. Spanner unifies data modeling, synchronous replication, timestamp-based concurrency control, and multi-versioned reads, defining a new class of globally consistent cloud databases that influenced successors such as Google Cloud Spanner and CockroachDB.